

Arquitectura de Computadores (AC)

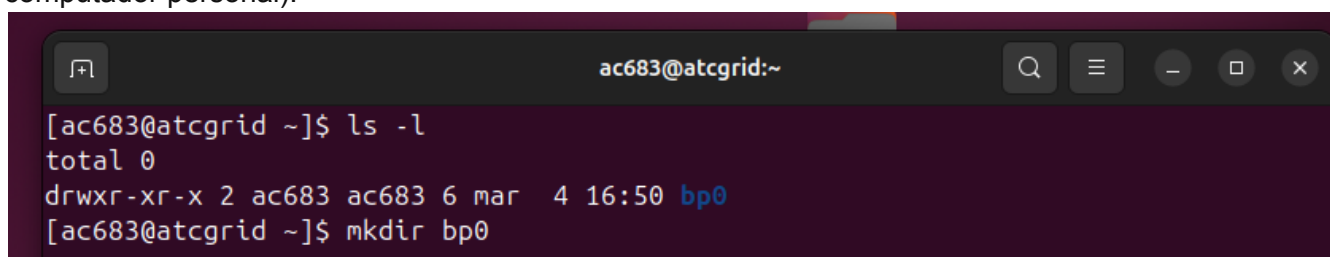
1.1.1 Cuaderno de prácticas.

1.1.2 Bloque Práctico 0. Entorno de programación.

Estudiante (nombre y apellidos): David Rodriguez Aparicio

Parte I. Ejercicios basados en los ejemplos del seminario práctico

Crear el directorio con nombre bp0 en atcgrid y en el PC (PC = PC del aula de prácticas o su computador personal).



```
ac683@atcgrid:~  
[ac683@atcgrid ~]$ ls -l  
total 0  
drwxr-xr-x 2 ac683 ac683 6 mar  4 16:50 bp0  
[ac683@atcgrid ~]$ mkdir bp0
```

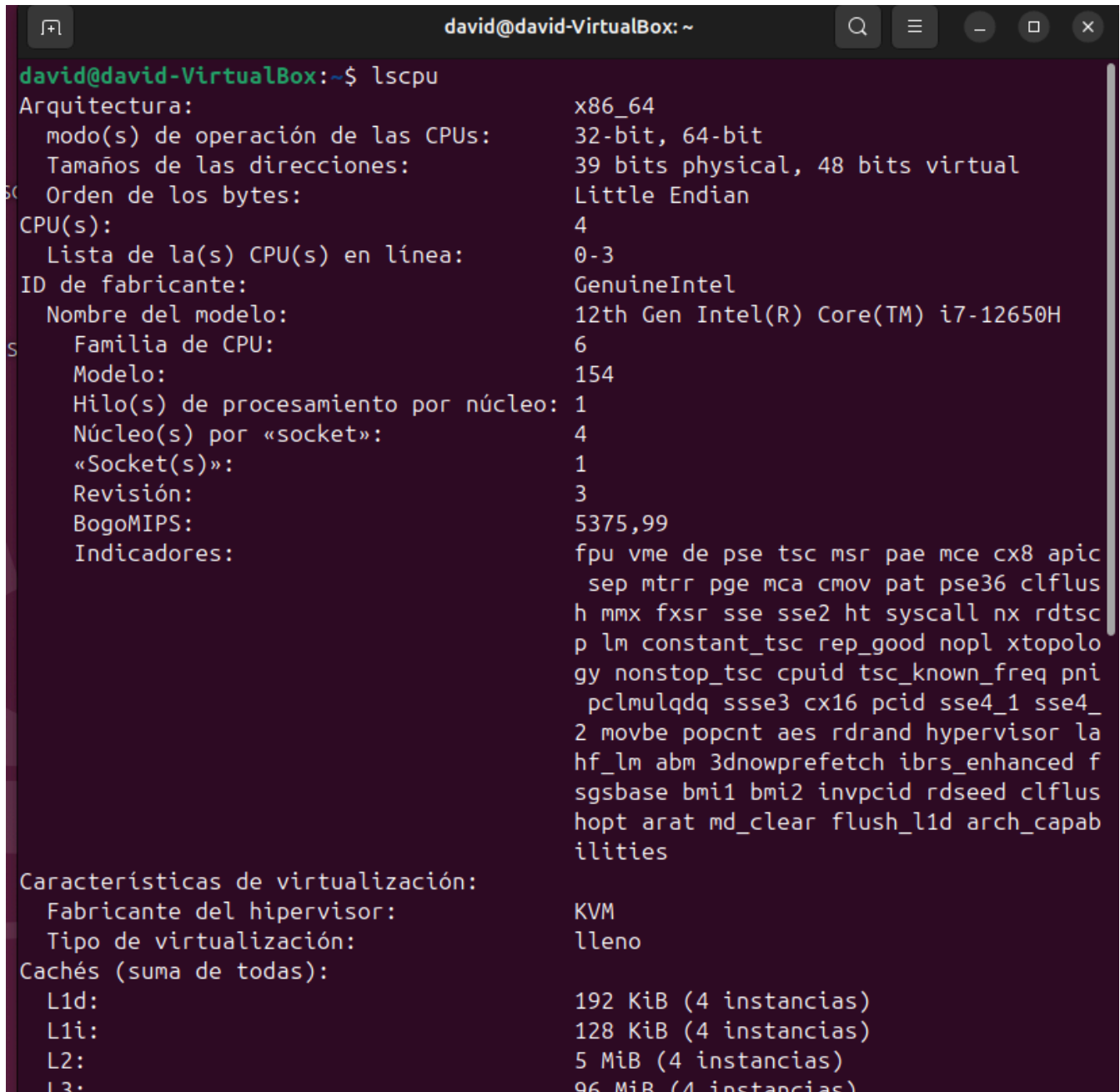
NOTA: En las prácticas se usa slurm como gestor de colas. Consideraciones a tener en cuenta:

- Slurm está configurado para asignar recursos a los procesos (llamados *tasks* en slurm) a nivel de core físico. Esto significa que por defecto slurm asigna **un core físico (dos lógicos) a un proceso**, para asignar x se debe usar con sbatch/srun la opción `--cpus-per-task=x (-cx)`.
- En slurm, por defecto, cpu se refiere a cores lógicos (ej. en la opción `-c`), si no se quieren usar cores lógicos hay que añadir la opción `--hint=nomultithread` a sbatch/srun. Para que con sbatch se tenga en cuenta `--hint=nomultithread` se debe usar srun dentro del script delante de los ejecutables.
- Usar siempre `--hint` para reducir los comportamientos confusos de slurm: `--hint=nomultithread` para no usar cores lógicos o, `--hint=multithread`, si se quiere usar cores lógicos.
- Para **asegurar** que solo se crea un proceso hay que incluir `--ntasks=1 (-n1)` en sbatch/srun.
- Para que un **nodo de cómputo** de atcgrid se asigne en exclusividad al trabajo de un usuario (enviado con srun o sbatch) hay que usar `--exclusive` con sbatch/srun (se recomienda no utilizarlo en los srun dentro de un script). Se asignará al trabajo todos los cores del nodo de cómputo independientemente del valor en `-c`. Tratará de usar todos los cores lógicos. Usar `--hint=nomultithread` para no usar cores lógicos y, si se quiere usar cores lógicos, añadir `--hint=multithread`. En las colas ac y ac4 sólo se asigna un nodo de cómputo a un trabajo.
- Los srun dentro de un *script* heredan las opciones fijadas en el sbatch que se usa para enviar el script a la cola (partición slurm).
- Las opciones de sbatch se pueden especificar también dentro del *script* (usando `#SBATCH`, ver ejemplos en el script del seminario, pero hay que tener cuidado porque no siempre el comportamiento de slurm es igual si la opción está dentro del script)
- Se recomienda escribir las órdenes directamente en la ventana de comandos (*shell*) en lugar de usar copiar y pegar.

1. Ejecutar `lscpu` en el PC, en `atcgrid4` (usar en este caso `-p ac4`) y en uno de los restantes nodos de cómputo (`atcgrid1`, `atcgrid2` o `atcgrid3`, usar en este caso `-p ac`).

(a) Mostrar con capturas de pantalla el resultado de estas ejecuciones.

RESPUESTA:



```
david@david-VirtualBox:~$ lscpu
Arquitectura:                x86_64
modo(s) de operación de las CPUs: 32-bit, 64-bit
Tamaños de las direcciones:  39 bits physical, 48 bits virtual
Orden de los bytes:          Little Endian
CPU(s):                      4
Lista de la(s) CPU(s) en línea: 0-3
ID de fabricante:            GenuineIntel
Nombre del modelo:           12th Gen Intel(R) Core(TM) i7-12650H
Familia de CPU:               6
Modelo:                       154
Hilo(s) de procesamiento por núcleo: 1
Núcleo(s) por «socket»:      4
«Socket(s)»:                  1
Revisión:                     3
BogoMIPS:                     5375,99
Indicadores:                  fpu vme de pse tsc msr pae mce cx8 apic
                              sep mtrr pge mca cmov pat pse36 clflush
                              h mmx fxsr sse sse2 ht syscall nx rdtsc
                              p lm constant_tsc rep_good nopl xtopolo
                              gy nonstop_tsc cpuid tsc_known_freq pni
                              pclmulqdq ssse3 cx16 pcid sse4_1 sse4_
                              2 movbe popcnt aes rdrand hypervisor la
                              hf_lm abm 3dnowprefetch ibrs_enhanced f
                              sgsbase bmi1 bmi2 invpcid rdseed clflush
                              hopt arat md_clear flush_l1d arch_capab
                              ilities

Características de virtualización:
Fabricante del hipervisor:    KVM
Tipo de virtualización:      lleno
Cachés (suma de todas):
L1d:                          192 KiB (4 instancias)
L1i:                          128 KiB (4 instancias)
L2:                            5 MiB (4 instancias)
L3:                          96 MiB (4 instancias)
```

```

[ac683@atcgrid ~]$ srun -p ac4 -n1 -c1 --hint=nomultithread lscpu
Arquitectura: x86_64
modo(s) de operación de las CPUs: 32-bit, 64-bit
Tamaños de las direcciones: 46 bits physical, 48 bits virtual
Orden de los bytes: Little Endian
CPU(s): 64
Lista de la(s) CPU(s) en línea: 0-63
ID de fabricante: GenuineIntel
Nombre del modelo: Intel(R) Xeon(R) Silver 4216 CPU @ 2.10GHz
Familia de CPU: 6
Modelo: 85
Hilo(s) de procesamiento por núcleo: 2
Núcleo(s) por «socket»: 16
«Socket(s)»: 2
Revisión: 7
CPU(s) scaling MHz: 98%
CPU MHz máx.: 3200,0000
CPU MHz mín.: 800,0000
BogoMIPS: 4200.00
Indicadores: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts
acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc art arch_perfmon pebs bts rep_good nopl xtopol
ogy nonstop_tsc cpuid aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid dca s
se4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb cat_l
3 cdp_l3 intel_ppin ssbd mba ibrs ibpb stibp ibrs_enhanced tpr_shadow flexpriority ept vpid ept_ad fsgsbase tsc_adjust bmi1 a
vx2 smep bmi2 erms invpcid cqm mpx rdt_a avx512f avx512dq rdseed adx snap clflushopt clwb intel_pt avx512cd avx512bw avx512vl
xsaveopt xsavec xgetbv1 xsaves cqm_llc cqm_occup_llc cqm_mbm_total cqm_mbm_local dtherm ida arat pln pts vnni pku ospke avx5
12_vnni md_clear flush_l1d arch_capabilities
Virtualización: VT-x
Caché L1d: 1 MiB (32 instancias)
Caché L1i: 1 MiB (32 instancias)
Caché L2: 32 MiB (32 instancias)
Caché L3: 44 MiB (2 instancias)
Modo(s) NUMA: 2
CPU(s) del nodo NUMA 0: 0-15,32-47
CPU(s) del nodo NUMA 1: 16-31,48-63
Vulnerabilidad Gather data sampling: Mitigation; Microcode
Vulnerabilidad Indirect target selection: Mitigation; Aligned branch/return thunks
Vulnerabilidad Itlb multihit: KVM: Mitigation: Split huge pages

```

```

[ac683@atcgrid ~]$ srun -p ac -n1 -c1 --hint=nomultithread lscpu
Arquitectura: x86_64
modo(s) de operación de las CPUs: 32-bit, 64-bit
Tamaños de las direcciones: 40 bits physical, 48 bits virtual
Orden de los bytes: Little Endian
CPU(s): 24
Lista de la(s) CPU(s) en línea: 0-23
ID de fabricante: GenuineIntel
Nombre del modelo: Intel(R) Xeon(R) CPU E5645 @ 2.40GHz
Familia de CPU: 6
Modelo: 44
Hilo(s) de procesamiento por núcleo: 2
Núcleo(s) por «socket»: 6
«Socket(s)»: 2
Revisión: 2
Aumento de frecuencia: activada
CPU(s) scaling MHz: 99%
CPU MHz máx.: 2401,0000
CPU MHz mín.: 1600,0000
BogoMIPS: 4799.85
Indicadores: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts
acpi mmx fxsr sse sse2 ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon pebs bts rep_good nopl xtopology non
stop_tsc cpuid aperfmperf pni dtes64 monitor ds_cpl vmx smx est tm2 ssse3 cx16 xtpr pdcm pcid dca sse4_1 sse4_2 popcnt lahf_l
m epb pti ssbd ibrs ibpb stibp tpr_shadow flexpriority ept vpid dtherm ida arat vnni flush_l1d
Virtualización: VT-x
Caché L1d: 384 KiB (12 instancias)
Caché L1i: 384 KiB (12 instancias)
Caché L2: 3 MiB (12 instancias)
Caché L3: 24 MiB (2 instancias)
Modo(s) NUMA: 2
CPU(s) del nodo NUMA 0: 0-5,12-17
CPU(s) del nodo NUMA 1: 6-11,18-23
Vulnerabilidad Gather data sampling: Not affected
Vulnerabilidad Indirect target selection: Not affected
Vulnerabilidad Itlb multihit: KVM: Mitigation: Split huge pages
Vulnerabilidad L1tf: Mitigation; PTE Inversion; VMX conditional cache flushes, SMT vulnerable
Vulnerabilidad Mds: Vulnerable; Clear CPU buffers attempted, no microcode; SMT vulnerable

```

(b) ¿Cuántos cores físicos y cuántos cores lógicos tiene atcgrid4?, ¿cuántos tienen atcgrid1, atcgrid2 y atcgrid3? y ¿cuántos tiene el PC? Razonar las respuestas

RESPUESTA:

Físicos : ATCGrid 1/2/3 -> 12 Cores | ATCGrid 4 -> 32 Cores | Mi PC -> 4 Cores

Lógicos : ATCGrid 1/2/3 -> 24 Cores | ATCGrid 4 -> 64 Cores | Mi PC -> 4 Cores

Cores Físicos = Cores por socket * socket

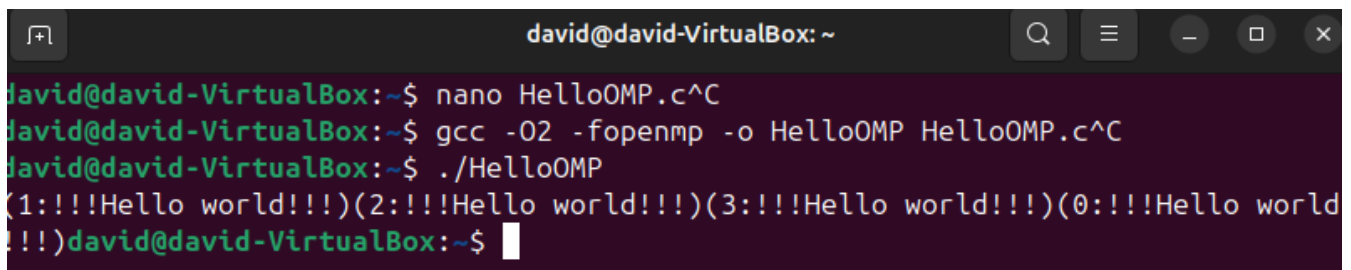
Cores Lógicos = Cores físicos * thread por core | CPUs

Con mi PC estamos viendo las características de la máquina virtual.

8. Compilar y ejecutar en el PC el código HelloOMP.c del seminario.

(a) Adjuntar capturas de pantalla que muestren la compilación y ejecución en el PC.

RESPUESTA:

A screenshot of a terminal window titled 'david@david-VirtualBox: ~'. The terminal shows the following commands and output:

```
david@david-VirtualBox:~$ nano HelloOMP.c^C
david@david-VirtualBox:~$ gcc -O2 -fopenmp -o HelloOMP HelloOMP.c^C
david@david-VirtualBox:~$ ./HelloOMP
(1:!!!Hello world!!!)(2:!!!Hello world!!!)(3:!!!Hello world!!!)(0:!!!Hello world
!!!)david@david-VirtualBox:~$
```

(b) Justificar el número de

“Hello world” que se imprimen en pantalla teniendo en cuenta la salida que devuelve lscpu en el PC.

RESPUESTA:

Salen 4 Hello Worlds, porque se ejecuta printf por cada uno de los hilos.

9. Copiar el ejecutable de HelloOMP.c que ha generado anteriormente y que se encuentra en el directorio ejer2 del PC al directorio ejer2 de su home en el *front-end* de atcgrid. Ejecutar este código en un nodo de cómputo de atcgrid (de 1 a 3) a través de cola ac del gestor de colas utilizando directamente en línea de comandos (no use ningún *script*):

```

depto: connection closed
david@david-VirtualBox:~$ scp HelloOMP ac683@atcgrid.ugr.es:~/ejer2/
ac683@atcgrid.ugr.es's password:
HelloOMP                               100% 16KB 1.1MB/s 00:00
david@david-VirtualBox:~$

```

(a) `srun --partition=ac --account=ac --ntasks=1 --cpus-per-task=12 --hint=nomultithread HelloOMP`

(Alternativa: `srun -p ac -A ac -n1 -c12 --hint=nomultithread HelloOMP`)

Adjuntar capturas de pantalla que muestren el envío a la cola de la ejecución y el resultado de esta ejecución tal y como la devuelve el gestor de colas.

RESPUESTA:

```

drwxr-xr-x 2 ac683 ac683 6 mar  4 16:50 bp0
drwxr-xr-x 2 ac683 ac683 6 mar  9 18:49 ejer2
[ac683@atcgrid ~]$ cd ejer2/
[ac683@atcgrid ejer2]$ ls -l
total 16
-rwxr-xr-x 1 ac683 ac683 16344 mar  9 18:51 HelloOMP
[ac683@atcgrid ejer2]$ chmod +x HelloOMP
[ac683@atcgrid ejer2]$ srun -p ac -A ac -n1 -c12 --hint=nomultithread ./HelloOMP
(2:!!!Hello world!!!)(4:!!!Hello world!!!)(11:!!!Hello world!!!)(5:!!!Hello world!!!)(3:!!!Hello world!!!)(7:!!
!!!Hello world!!!)(8:!!!Hello world!!!)(0:!![a[a[a[a[a[a[a[ac683[ac6[ac6[ac683[ac683[ac6[a[a[a[a[a[a[a
[ac683@atcgrid ejer2]$

```

(b) `srun -p ac -A ac -n1 -c24 HelloOMP`

Adjuntar capturas de pantalla que muestren el envío a la cola de la ejecución y el resultado de esta ejecución tal y como la devuelve el gestor de colas.

RESPUESTA:

```

[ac683@atcgrid ejer2]$ srun -p ac -A ac -n1 -c24 ./HelloOMP
(2:!!!Hello world!!!)(11:!!!Hello world!!!)(5:!!!Hello world!!!)(8:!!!Hello world!!!)(15:!!!Hello world!!!)(23
:!!!Hello world!!!)(7:!!!Hello world!!!)(19:!!!Hello world!!!)(12:!!!Hello world!!!)(20:!!!Hello world!!!)(6:!!
!!!Hello world!!!)(0:!!!Hello world!!!)(18:!!!Hello world!!!)(9:!!!Hello world!!!)(10:!!!Hello world!!!)(13:!!!
Hello world!!!)(3:!!!Hello world!!!)(16:!!!Hello world!!!)(4:!!!Hello world!!!)(22:!!!Hello world!!!)(17:!!!He
llo world!!!)(1:!!!Hello world!!!)(14:!!!Hello world!!!)(21:!!!Hello world!!!)[ac683@atcgrid ejer2]$ ^C

```

(c) `srun -n1 HelloOMP`

Adjuntar capturas de pantalla que muestren el envío a la cola de la ejecución y el resultado de esta ejecución tal y como la devuelve el gestor de colas. ¿Qué partición (cola) se está usando?

RESPUESTA:

```

... world!!!)(1:!!!Hello world!!!)(14:!!!Hello world!!!)(21:!!!Hello wo
[ac683@atcgrid ejer2]$ srun -n1 ./HelloOMP
(1:!!!Hello world!!!)(0:!!!Hello world!!!)[ac683@atcgrid ejer2]$ ^C
[ac683@atcgrid ejer2]$

```

(d) ¿Qué orden `srun` usaría para que HelloOMP utilice todos los cores físicos de atcgrid4 (se debe imprimir un único mensaje desde cada uno de ellos)?

`srun -p ac4 -A ac -n1 -c32 --hint=nomultithread ./HelloOMP`

10. Modificar en su PC HelloOMP.c para que se imprima “world” en un printf distinto al usado para “Hello”. En ambos printf se debe imprimir el identificador del thread que escribe en pantalla (poner los dos printf entre llaves {}). Nombrar al código resultante HelloOMP2.c. Compilar este nuevo código en el PC y ejecutarlo. Copiar el fichero ejecutable resultante al front-end de atcgrid (directorio ejer4). Ejecutar el código en un nodo de cómputo de atcgrid usando el *script* script_helloomp.sh del seminario (el nombre del ejecutable en el script debe ser HelloOMP2).

```
[ac683@atcgrid ~]$ cp ./ejer2/HelloOMP2 ./ejer4/
[ac683@atcgrid ~]$ cd ./ejer4/
[ac683@atcgrid ejer4]$ ls -l
total 20
-rwxr-xr-x 1 ac683 ac683 17912 mar  9 19:14 HelloOMP2
[ac683@atcgrid ejer4]$ chmod +x HelloOMP2
[ac683@atcgrid ejer4]$ nano script_helloomp.sh
[ac683@atcgrid ejer4]$ ls -l
total 24
-rwxr-xr-x 1 ac683 ac683 17912 mar  9 19:14 HelloOMP2
-rw-r--r-- 1 ac683 ac683  580 mar  9 19:15 script_helloomp.sh
[ac683@atcgrid ejer4]$ chmod +x script_helloomp.sh
[ac683@atcgrid ejer4]$
```

```
GNU nano 5.6.1                ../ejer2/HelloOMP2.c
#include <stdio.h>
#include <omp.h>

int main(void) {
    #pragma omp parallel
    {
        printf("(%d: Hello)\n", omp_get_thread_num());
        printf("(%d: world)\n", omp_get_thread_num());
    }
    return 0;
}
```

```
GNU nano 5.6.1                script_helloomp.sh
#!/bin/bash
#SBATCH --job-name=helloOMP2
#SBATCH --partition=ac
#SBATCH --account=ac

echo "Id. usuario del trabajo: $SLURM_JOB_USER"
echo "Id. del trabajo: $SLURM_JOBID"
echo "Nombre del trabajo especificado por usuario: $SLURM_JOB_NAME"
echo "Directorio de trabajo: $SLURM_SUBMIT_DIR"
echo "Cola: $SLURM_JOB_PARTITION"
echo "Nodo que ejecuta este trabajo: $SLURM_JOB_NODELIST"
echo "Nº de nodos asignados al trabajo: $SLURM_JOB_NUM_NODES"
echo "Nº CPUs disponibles para el trabajo en el nodo: $SLURM_JOB_CPUS_PER_NODE"

echo -e "\nEjecución de HelloOMP2:\n"
srun ./HelloOMP2
```

(a) Utilizar: `sbatch -n1 -c12 --exclusive script_helloomp.sh`. Adjuntar capturas de pantalla que muestren el nuevo código, la compilación, el envío a la cola de la ejecución y el resultado de esta ejecución tal y como la devuelve el gestor de colas.

RESPUESTA:

```
[ac683@atcgrid ejer4]$ sbatch -n1 -c12 --exclusive --hint=nomultithread script_helloomp.sh
Submitted batch job 44884
[ac683@atcgrid ejer4]$ ls -l
total 36
-rwxr-xr-x 1 ac683 ac683 17912 mar  9 19:14 HelloOMP2
-rwxr-xr-x 1 ac683 ac683  580 mar  9 19:15 script_helloomp.sh
-rw-r--r-- 1 ac683 ac683  337 mar  9 19:16 slurm-44882.out
-rw-r--r-- 1 ac683 ac683  337 mar  9 19:17 slurm-44883.out
-rw-r--r-- 1 ac683 ac683  337 mar  9 19:17 slurm-44884.out
[ac683@atcgrid ejer4]$ cat slurm-44884.out
Id. usuario del trabajo: ac683
Id. del trabajo: 44884
Nombre del trabajo especificado por usuario: helloOMP2
Directorio de trabajo: /home/ac683/ejer4
Cola: ac
Nodo que ejecuta este trabajo: atcgrid1
Nº de nodos asignados al trabajo: 1
Nº CPUs disponibles para el trabajo en el nodo: 12

Ejecución de HelloOMP2:

(0: Hello)
(0: world)
[ac683@atcgrid ejer4]$
```

(b) ¿Qué nodo de cómputo de atcgrid ha ejecutado el *script*? Explicar cómo ha obtenido esta información.

RESPUESTA:

De las variables de nodo NODELIST y PARTITION, que hemos usado en el script. (atcgrid1)

(c) ¿Qué órdenes para el gestor de colas slurm incluye el *script*? Explicar cómo ha obtenido esta información.

RESPUESTA:

job_name, partition y account, además n1, c12, exclusive y nomultithread.

(d) Haga los cambios necesarios en el *script* para que se utilice atcgrid4. Comentar los cambios realizados y los motivos por los que se han hecho.

RESPUESTA:

```
GNU nano 5.6.1
c#!/bin/bash
#SBATCH --job-name=helloOMP2
#SBATCH --partition=ac
#SBATCH --account=ac4
```

y podríamos cambiar c12, por c32, por ejemplo.

NOTA: Utilizar siempre con sbatch las opciones -n1, -c, --exclusive y --hint (--hint=nomultithread para usar cores físicos y no lógicos, o --hint=multithread para usar cores lógicos). Utilizar siempre con srun, si se usa fuera de un script, las opciones -n1 y -c y --hint. Recordar que los srun dentro de un *script* heredan las opciones incluidas en el sbatch que se usa para enviar el *script* a la cola slurm. Se recomienda usar sbatch en lugar de srun para enviar trabajos a ejecutar a través slurm porque éste último deja bloqueada la ventana hasta que termina la ejecución, mientras que usando sbatch la ejecución se realiza en segundo plano.

Parte II. Resto de ejercicios

11. Generar en el PC el ejecutable del código fuente C SumaVectores.c para vectores locales (para ello antes de compilar debe descomentar la definición de VECTOR_LOCAL y comentar las definiciones de VECTOR_GLOBAL y VECTOR_DYNAMIC). El comentario inicial del código muestra la orden para compilar (siempre hay que usar -O2). Incorporar volcados de pantalla que demuestren la compilación y la ejecución correcta del código en el PC.

RESPUESTA:

```
david@david-VirtualBox: ~
david@david-VirtualBox:~$ gcc -O2 SumaVectores.c -o SumaVectores -lrt^C
david@david-VirtualBox:~$ ./SumaVectores 6
Tiempo(seg.):0.000000158 / Tamaño Vectores:6
/ V1[0]+V2[0]=V3[0](0.600000+0.600000=1.200000) /
/ V1[1]+V2[1]=V3[1](0.700000+0.500000=1.200000) /
/ V1[2]+V2[2]=V3[2](0.800000+0.400000=1.200000) /
/ V1[3]+V2[3]=V3[3](0.900000+0.300000=1.200000) /
/ V1[4]+V2[4]=V3[4](1.000000+0.200000=1.200000) /
/ V1[5]+V2[5]=V3[5](1.100000+0.100000=1.200000) /
david@david-VirtualBox:~$ ./SumaVectores 9
Tiempo(seg.):0.000000093 / Tamaño Vectores:9
/ V1[0]+V2[0]=V3[0](0.267572+0.274146=0.541718) /
/ V1[1]+V2[1]=V3[1](0.552233+0.761749=1.313982) /
/ V1[2]+V2[2]=V3[2](0.194918+0.183993=0.378911) /
/ V1[3]+V2[3]=V3[3](0.275084+0.187570=0.462654) /
/ V1[4]+V2[4]=V3[4](0.841450+0.399590=1.241040) /
/ V1[5]+V2[5]=V3[5](0.729089+0.690100=1.419189) /
/ V1[6]+V2[6]=V3[6](0.553293+0.272071=0.825365) /
/ V1[7]+V2[7]=V3[7](0.787806+0.564991=1.352798) /
/ V1[8]+V2[8]=V3[8](0.137899+0.377527=0.515426) /
david@david-VirtualBox:~$ ./SumaVectores 20
Tiempo(seg.):0.000000087 / Tamaño Vectores:20 / V1[0]+V2[0]=V3[0](0.812816+0.661365=1.47418
1) // V1[19]+V2[19]=V3[19](0.612017+0.036001=0.648018) /
david@david-VirtualBox:~$ ./SumaVectores 100
Tiempo(seg.):0.000000199 / Tamaño Vectores:100 / V1[0]+V2[0]=V3[0](0.520839+0.707270=1.22810
9) // V1[99]+V2[99]=V3[99](0.068927+0.913846=0.982774) /
david@david-VirtualBox:~$
```

12. En el código SumaVectores.c se utiliza la función `clock_gettime()` para obtener el tiempo de ejecución del trozo de código que calcula la suma de vectores. El código se imprime la variable `ncgt`,
(a) ¿Qué contiene esta variable?

RESPUESTA:

El tiempo transcurrido entre la llamada 1 y 2 de `clock_gettime()`, que al final es el tiempo de suma de los vectores.

- (b) ¿En qué estructura de datos devuelve `clock_gettime()` la información de tiempo (indicar el tipo de estructura de datos, describir la estructura de datos, e indicar los tipos de datos que usa)?

RESPUESTA:

En una estructura llamada `timespec`, que contiene 2 valores, el primero los segundos y en el segundo los nanosegundos.

- (c) ¿Qué información devuelve exactamente la función `clock_gettime()` en la estructura de datos descrita en el apartado (b)? ¿qué representan/significan los valores numéricos que devuelve?

RESPUESTA:

Devuelve un instante del reloj del sistema, el del inicio y el del final.

13. Rellenar una tabla como la Tabla 1 en una hoja de cálculo con los tiempos de ejecución del código SumaVectores.c para vectores locales, globales y dinámicos (se pueden obtener errores en tiempo de ejecución o de compilación, ver ejercicio 9). Obtener estos resultados usando *scripts* (partir del *script* que hay en el seminario). Debe haber una tabla para un nodo de cómputo de atcgrid con procesador Intel Xeon E5645 y otra para su PC en la hoja de cálculo. En la columna “Bytes de un vector” hay que poner el total de bytes reservado para un vector. (NOTA: Se recomienda usar en la hoja de cálculo el mismo separador para decimales que usan los códigos al imprimir “.”. Este separador se puede modificar en la hoja de cálculo.)

RESPUESTA:

Tabla 1 . Copiar la tabla de la hoja de cálculo utilizada. El número de componentes se va duplicando.

Nº de Componentes	Bytes de un vector	Tiempo para vect. locales	Tiempo para vect. globales	Tiempo para vect. dinámicos
65536	524288	0.000173814	0.000752445	0.000257804
131072	1.048.576	0.000083584	0.000350412	0.000300347
262144	2.097.152	0.000380395	0.000649729	0.001012579
524288	4.194.304	Error	0.001605668	0.001225333
1048576	8.388.608	Error	0.003171307	0.016531839
2097152	16.777.216	Error	0.016910109	0.039448404
4194304	33.554.432	Error	0.043288022	0.071917691
8388608	67.108.864	Error	0.026220982	0.139586110
16777216	134.217.728	Error	0.191094357	0.246585285
33554432	268.435.456	Error	0.603188969	0.254074570
67108864	536.870.912	Error	0.096048275	0.983490162

ATC GRID

Nº de Componentes	Bytes de un vector	Tiempo para vect. locales	Tiempo para vect. globales	Tiempo para vect. dinámicos
65536	524288	0.000093612	0.000300907	0.000348160
131072	1048576	0.000187421	0.000636347	0.000657871
262144	2097152	0.000394647	0.000734500	0.001286566
524288	4194304	Error	0.001764002	0.002090404
1048576	8388608	Error	0.004633470	0.004537097
2097152	16777216	Error	0.008887926	0.008437304
4194304	33554432	Error	0.016765819	0.016108644
8388608	67108864	Error	0.032404105	0.031267677
16777216	134217728	Error	0.065229903	0.062789681
33554432	268435456	Error	0.124480027	0.123799229
67108864	536870912	Error	0.124379457	0.241525716

```
ac683@atcgrid:~/ejerTablas
GNU nano 5.6.1 medir_atcgrid.sh Modificado
#!/bin/bash
#SBATCH --job-name=sumavec
#SBATCH --partition=ac
#SBATCH --account=ac

sizes="65536 131072 262144 524288 1048576 2097152 4194304 8388608 16777216 33554432"

for N in $sizes
do
    echo "==== N=$N ====="
    echo "LOCAL"
    srun -n1 -c1 --hint=nomultithread ./SumaVectores_local $N
    echo "GLOBAL"
    srun -n1 -c1 --hint=nomultithread ./SumaVectores_global $N
    echo "DYNAMIC"
    srun -n1 -c1 --hint=nomultithread ./SumaVectores_dynamic $N
    echo
done
```

```
ac6830atcgrid ejerTablas] sbatch -n1 -c1 --exclusive --hint=nomultithread medtr_atcgrid.sh%
[ac6830atcgrid ejerTablas] cat slurm-44922.out
===== N=65536 =====
LOCAL
Ttempo(seg.):0.000093612 / Tamaño Vectores:65536 / V1[0]+V2[0]=V3[0](0.004440+0.430778=1.335218) // V1[65535]+V2[65535]=V3[65535](0.032125+0.505946=0.538071) /
GLOBAL
Ttempo(seg.):0.000300907 / Tamaño Vectores:65536 / V1[0]+V2[0]=V3[0](0.775243+0.135368=0.910611) // V1[65535]+V2[65535]=V3[65535](0.888373+0.190638=1.079011) /
DYNAMIC
Ttempo(seg.):0.000348160 / Tamaño Vectores:65536 / V1[0]+V2[0]=V3[0](0.775243+0.135368=0.910611) // V1[65535]+V2[65535]=V3[65535](0.888373+0.190638=1.079011) /
===== N=131072 =====
LOCAL
Ttempo(seg.):0.000187421 / Tamaño Vectores:131072 / V1[0]+V2[0]=V3[0](0.775243+0.135368=0.910611) // V1[131071]+V2[131071]=V3[131071](0.115803+0.287052=0.402855) /
GLOBAL
Ttempo(seg.):0.000636347 / Tamaño Vectores:131072 / V1[0]+V2[0]=V3[0](0.775243+0.135368=0.910611) // V1[131071]+V2[131071]=V3[131071](0.115803+0.287052=0.402855) /
DYNAMIC
Ttempo(seg.):0.000657871 / Tamaño Vectores:131072 / V1[0]+V2[0]=V3[0](0.775243+0.135368=0.910611) // V1[131071]+V2[131071]=V3[131071](0.115803+0.287052=0.402855) /
===== N=262144 =====
LOCAL
Ttempo(seg.):0.000394647 / Tamaño Vectores:262144 / V1[0]+V2[0]=V3[0](0.775243+0.135368=0.910611) // V1[262143]+V2[262143]=V3[262143](0.170028+0.435690=0.605717) /
GLOBAL
Ttempo(seg.):0.000734500 / Tamaño Vectores:262144 / V1[0]+V2[0]=V3[0](0.775243+0.135368=0.910611) // V1[262143]+V2[262143]=V3[262143](0.170028+0.435690=0.605717) /
DYNAMIC
Ttempo(seg.):0.001286566 / Tamaño Vectores:262144 / V1[0]+V2[0]=V3[0](0.775243+0.135368=0.910611) // V1[262143]+V2[262143]=V3[262143](0.170028+0.435690=0.605717) /
===== N=524288 =====
LOCAL
slurm: error: atcgrid1: task 0: Segmentation fault (core dumped)
GLOBAL
Ttempo(seg.):0.001764002 / Tamaño Vectores:524288 / V1[0]+V2[0]=V3[0](0.646045+0.839958=1.486003) // V1[524287]+V2[524287]=V3[524287](0.015951+0.033746=0.049698) /
DYNAMIC
Ttempo(seg.):0.002090404 / Tamaño Vectores:524288 / V1[0]+V2[0]=V3[0](0.646045+0.839958=1.486003) // V1[524287]+V2[524287]=V3[524287](0.015951+0.033746=0.049698) /
===== N=1048576 =====
LOCAL
slurm: error: atcgrid1: task 0: Segmentation fault (core dumped)
GLOBAL
Ttempo(seg.):0.004633470 / Tamaño Vectores:1048576 / V1[0]+V2[0]=V3[0](0.646045+0.839958=1.486003) // V1[1048575]+V2[1048575]=V3[1048575](0.599062+0.045289=0.644351) /
DYNAMIC
Ttempo(seg.):0.004537007 / Tamaño Vectores:1048576 / V1[0]+V2[0]=V3[0](0.516847+0.544549=1.061396) // V1[1048575]+V2[1048575]=V3[1048575](0.920520+0.000367=0.920887) /
===== N=2097152 =====
LOCAL
slurm: error: atcgrid1: task 0: Segmentation fault (core dumped)
GLOBAL
Ttempo(seg.):0.008887026 / Tamaño Vectores:2097152 / V1[0]+V2[0]=V3[0](0.516847+0.544549=1.061396) // V1[2097151]+V2[2097151]=V3[2097151](0.409007+0.150407=0.559413) /
DYNAMIC
Ttempo(seg.):0.008437304 / Tamaño Vectores:2097152 / V1[0]+V2[0]=V3[0](0.516847+0.544549=1.061396) // V1[2097151]+V2[2097151]=V3[2097151](0.409007+0.150407=0.559413) /
```

```

===== N=4194304 =====
LOCAL
srun: error: atcgrid1: task 0: Segmentation fault (core dumped)
GLOBAL
Tiempo(seg.):0.016765819 / Tamaño Vectores:4194304 / V1[0]+V2[0]=V3[0](0.387649+0.249139=0.636789) // V1[4194303]+V2[4194303]=V3[4194303](0.033611+0.958297=0.991908) /
DYNAMIC
Tiempo(seg.):0.016108644 / Tamaño Vectores:4194304 / V1[0]+V2[0]=V3[0](0.387649+0.249139=0.636789) // V1[4194303]+V2[4194303]=V3[4194303](0.033611+0.958297=0.991908) /

===== N=8388608 =====
LOCAL
srun: error: atcgrid1: task 0: Segmentation fault (core dumped)
GLOBAL
Tiempo(seg.):0.032404105 / Tamaño Vectores:8388608 / V1[0]+V2[0]=V3[0](0.258452+0.953730=1.212182) // V1[8388607]+V2[8388607]=V3[8388607](0.525813+0.144391=0.670204) /
DYNAMIC
Tiempo(seg.):0.031267677 / Tamaño Vectores:8388608 / V1[0]+V2[0]=V3[0](0.258452+0.953730=1.212182) // V1[8388607]+V2[8388607]=V3[8388607](0.525813+0.144391=0.670204) /

===== N=16777216 =====
LOCAL
srun: error: atcgrid1: task 0: Segmentation fault (core dumped)
GLOBAL
Tiempo(seg.):0.065229903 / Tamaño Vectores:16777216 / V1[0]+V2[0]=V3[0](0.129254+0.658320=0.787574) // V1[16777215]+V2[16777215]=V3[16777215](0.854776+0.157204=1.011980) /
DYNAMIC
Tiempo(seg.):0.062789681 / Tamaño Vectores:16777216 / V1[0]+V2[0]=V3[0](0.129254+0.658320=0.787574) // V1[16777215]+V2[16777215]=V3[16777215](0.854776+0.157204=1.011980) /

===== N=33554432 =====
LOCAL
srun: error: atcgrid1: task 0: Segmentation fault (core dumped)
GLOBAL
Tiempo(seg.):0.124480027 / Tamaño Vectores:33554432 / V1[0]+V2[0]=V3[0](0.000056+0.362911=0.362967) // V1[33554431]+V2[33554431]=V3[33554431](0.560384+0.464079=1.024463) /
DYNAMIC
Tiempo(seg.):0.123799229 / Tamaño Vectores:33554432 / V1[0]+V2[0]=V3[0](0.870859+0.067501=0.938360) // V1[33554431]+V2[33554431]=V3[33554431](0.731452+0.026579=0.758031) /

===== N=67108864 =====
LOCAL
srun: error: atcgrid1: task 0: Segmentation fault (core dumped)
GLOBAL
Tiempo(seg.):0.124379457 / Tamaño Vectores:33554432 / V1[0]+V2[0]=V3[0](0.612463+0.476682=1.089146) // V1[33554431]+V2[33554431]=V3[33554431](0.073587+0.151579=0.225166) /
DYNAMIC
Tiempo(seg.):0.241525716 / Tamaño Vectores:67108864 / V1[0]+V2[0]=V3[0](0.483266+0.181273=0.664539) // V1[67108863]+V2[67108863]=V3[67108863](0.376236+0.015330=0.391565) /

```

8. Con ayuda de la hoja de cálculo representar **en una misma gráfica** los tiempos de ejecución obtenidos en atcgrid y en su PC para vectores locales, globales y dinámicos (eje y) en función del tamaño en bytes de un vector (por tanto, los valores de la segunda columna de la tabla, que están en escala logarítmica, deben estar en el eje x). Utilizar escala logarítmica en el eje de ordenadas (eje y). ¿Hay diferencias en los tiempos de ejecución?

RESPUESTA: Si claro, a medida que van aumentando los vectores aumentan los tiempos, y hay cambios entre mi PC y el atcgrid

14. Contestar a las siguientes preguntas:

(a) Cuando se usan vectores locales, ¿se obtiene error para alguno de los tamaños?, ¿a qué cree que es debido lo que ocurre? (Incorporar volcados de pantalla)

RESPUESTA: Al ser locales se almacenan en pila y llega un momento que la pila no aguanta ese tamaño de datos

(b) Cuando se usan vectores globales, ¿se obtiene error para alguno de los tamaños?, ¿a qué cree que es debido lo que ocurre? (Incorporar volcados de pantalla)

RESPUESTA: No aparece error porque soporta esa cantidad, pero hay un límite.

(c) Cuando se usan vectores dinámicos, ¿se obtiene error para alguno de los tamaños?, ¿a qué cree que es debido lo que ocurre? (Incorporar volcados de pantalla)

RESPUESTA: En este caso se reserva mediante malloc, por lo que usa el heap y no la pila, en este caso tampoco hemos obtenido errores

15. (a) ¿Cuál es el máximo valor que se puede almacenar en la variable N teniendo en cuenta su tipo? Razonar respuesta.

RESPUESTA: $2^{32} - 1$ (4294967295)

(b) Modificar el código fuente C (en el PC) para que el límite de los vectores cuando **se declaran** como variables globales sea igual al máximo número que se puede almacenar en la variable N y generar el ejecutable. ¿Qué ocurre? ¿A qué es debido? (Incorporar volcados de pantalla que muestren lo que ocurre)

RESPUESTA: No termina de compilar, ya que son numeros y vectores tan grandes que la memoria global no lo soporta

Enlaces de interés:

<https://unix.stackexchange.com/questions/468766/understanding-output-of-lscpu>

<https://www.geeksforgeeks.org/memory-layout-of-c-program/>

Listado 1. Código C que suma dos vectores. Se generan aleatoriamente las componentes para vectores de tamaño mayor que 8 y se imprimen todas las componentes para vectores menores que 10.

```
/* SumaVectoresC.c
Suma de dos vectores: v3 = v1 + v2

Para compilar usar (-lrt: real time library, no todas las versiones de gcc necesitan que se incluya -lrt):
gcc -O2 SumaVectores.c -o SumaVectores -lrt
gcc -O2 -S SumaVectores.c -lrt //para generar el código ensamblador

Para ejecutar use: SumaVectoresC longitud
*/

#include <stdlib.h> // biblioteca con funciones atoi(), malloc() y free()
#include <stdio.h> // biblioteca donde se encuentra la función printf()
#include <time.h> // biblioteca donde se encuentra la función clock_gettime()

//Sólo puede estar definida una de las tres constantes VECTOR_ (sólo uno de los ...
//tres defines siguientes puede estar descomentado):
//define VECTOR_LOCAL // descomentar para que los vectores sean variables ...
// locales (si se supera el tamaño de la pila se ...
// generará el error "Violación de Segmento")
//define VECTOR_GLOBAL // descomentar para que los vectores sean variables ...
// globales (su longitud no estará limitada por el ...
// tamaño de la pila del programa)
#define VECTOR_DYNAMIC // descomentar para que los vectores sean variables ...
// dinámicas (memoria reutilizable durante la ejecución)
#ifdef VECTOR_GLOBAL
#define MAX 33554432 //2^25
double v1[MAX], v2[MAX], v3[MAX];
#endif
```

```

int main(int argc, char** argv){

    int i;
    struct timespec cgt1,cgt2; double ncgt; //para tiempo de ejecución

    //Leer argumento de entrada (nº de componentes del vector)
    if (argc<2){
        printf("Faltan nº componentes del vector\n");
        exit(-1);
    }

    unsigned int N = atoi(argv[1]); // Máximo N =2^32-1=4294967295 (sizeof(unsigned int) = 4 B)
    #ifdef VECTOR_LOCAL
    double v1[N], v2[N], v3[N]; // Tamaño variable local en tiempo de ejecución ...
        // disponible en C a partir de actualización C99
    #endif
    #ifdef VECTOR_GLOBAL
    if (N>MAX) N=MAX;
    #endif
    #ifdef VECTOR_DYNAMIC
    double *v1, *v2, *v3;
    v1 = (double*) malloc(N*sizeof(double)); // malloc necesita el tamaño en bytes
    v2 = (double*) malloc(N*sizeof(double)); //si no hay espacio suficiente malloc devuelve NULL
    v3 = (double*) malloc(N*sizeof(double));
        if ( (v1==NULL) || (v2==NULL) || (v3==NULL) ){
            printf("Error en la reserva de espacio para los vectores\n");
            exit(-2);
        }
    #endif

    //Inicializar vectores
    if (N < 9)
        for (i = 0; i < N; i++)
        {
            v1[i] = N * 0.1 + i * 0.1;
            v2[i] = N * 0.1 - i * 0.1;
        }
    else
    {
        srand48(time(0));
        for (i = 0; i < N; i++)
        {
            v1[i] = drand48();
            v2[i] = drand48(); //printf("%d:%f,%f",i,v1[i],v2[i]);
        }
    }

    clock_gettime(CLOCK_REALTIME,&cgt1);
    //Calcular suma de vectores
    for(i=0; i<N; i++)
        v3[i] = v1[i] + v2[i];

    clock_gettime(CLOCK_REALTIME,&cgt2);
    ncgt=(double) (cgt2.tv_sec-cgt1.tv_sec)+
        (double) ((cgt2.tv_nsec-cgt1.tv_nsec)/(1.e+9));

    //Imprimir resultado de la suma y el tiempo de ejecución

```

```
if (N<10) {
printf("Tiempo(seg.):%11.9ft / Tamaño Vectores:%lu\n",ncgt,N);
for(i=0; i<N; i++)
printf("/ V1[%d]+V2[%d]=V3[%d](%8.6f+%8.6f=%8.6f) \n",
i,i,i,v1[i],v2[i],v3[i]);
}
else
printf("Tiempo(seg.):%11.9ft / Tamaño Vectores:%u\t/ V1[0]+V2[0]=V3[0](%8.6f+%8.6f=%8.6f) //
V1[%d]+V2[%d]=V3[%d](%8.6f+%8.6f=%8.6f) \n",
ncgt,N,v1[0],v2[0],v3[0],N-1,N-1,N-1,v1[N-1],v2[N-1],v3[N-1]);

#ifdef VECTOR_DYNAMIC
free(v1); // libera el espacio reservado para v1
free(v2); // libera el espacio reservado para v2
free(v3); // libera el espacio reservado para v3
#endif
return 0;
}
```